API Security Risk Analyzer

Project Documentation

Version 1.0.0

Project	API Security Risk Analyzer
Type	Full-Stack Web Application
Internship	Cybersecurity Internship — Task 3
Author	Utpal
Tech Stack	Python · FastAPI · React · Vite · Tailwind CSS
Date	25 April 2026

1. Project Overview

The API Security Risk Analyzer is a full-stack web application that performs automated, read-only security audits on public REST APIs and produces a professional downloadable report. It was built as the deliverable for Cybersecurity Internship Task 3 and demonstrates real-world application security consulting skills — specifically, the workflow that professional AppSec agencies follow when auditing a SaaS client's API surface.

The tool codifies six security checks drawn from the OWASP API Security Top 10 (2023 edition). Each check sends one or more carefully scoped HTTP requests to a target API, inspects the response for evidence of a vulnerability, and produces a structured Finding object that flows through to the generated report. The entire process runs in under 30 seconds and requires no manual steps once a target is selected.

1.1 Purpose and Goals

- Automate the most time-consuming parts of an API security assessment — endpoint testing, header inspection, and authorization checks.
- Produce a client-ready security report (PDF and DOCX) that matches the format used by professional cybersecurity agencies.
- Demonstrate the OWASP API Security Top 10 vulnerabilities in a concrete, hands-on project.
- Provide a working codebase that can be extended with additional checks, target APIs, or report formats.

1.2 Ethical Scope

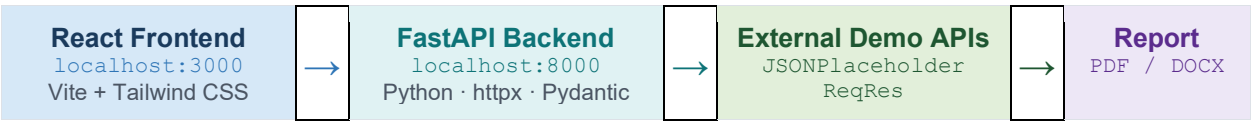
The tool is scoped exclusively to public demo APIs designed for testing: JSONPlaceholder (jsonplaceholder.typicode.com) and ReqRes (reqres.in). The backend enforces an allow-list of approved domains and rejects any request targeting a URL outside this list with HTTP 403. All tests are read-only; no data is modified, created, or deleted at any point.

> ⚠ **WARNING**
> This tool must not be pointed at any private, proprietary, or production API. Testing systems without explicit written permission is illegal under the IT Act 2000 (India) and equivalent legislation in other jurisdictions.

2. System Architecture

The system follows a clean client-server architecture. The React frontend communicates exclusively with the FastAPI backend over a well-defined REST interface. The backend is responsible for all outbound HTTP traffic to external APIs — the frontend never calls JSONPlaceholder or ReqRes directly.

2.1 High-Level Architecture Diagram



2.2 Directory Structure

```
api-security-analyzer/
├── backend/
│   ├── main.py                  # FastAPI app, CORS, route definitions
│   ├── analyzer/
│   │   ├── engine.py            # Orchestrates all checks, builds AnalysisReport
│   │   ├── models.py            # Pydantic data models (Finding, Severity,
Report)
│   │   └── checks/
│   │       ├── auth_check.py        # F-01: Unauthenticated access
│   │       ├── bola_check.py        # F-02: Broken Object Level Authorization
│   │       ├── data_exposure.py     # F-03: Excessive data exposure
│   │       ├── rate_limit_check.py  # F-04: Rate limiting detection
│   │       ├── header_check.py      # F-05: Security header analysis
│   │       └── token_check.py       # F-06: Auth token enforcement
│   ├── report/
│   │   └── generator.py         # PDF (ReportLab) + DOCX (python-docx) generation
│   └── requirements.txt
├── frontend/
│   ├── src/
│   │   ├── App.jsx              # Root component, routing
│   │   ├── api.js               # Axios instance, all backend calls
│   │   └── components/
│   │       ├── Dashboard.jsx        # Main layout, state management
│   │       ├── ApiSelector.jsx      # Target API selection cards
│   │       ├── FindingsTable.jsx    # Expandable findings table + filters
│   │       ├── SeverityBadge.jsx    # Color-coded severity label
│   │       ├── ProgressTracker.jsx  # Real-time scan progress
│   │       └── ReportDownload.jsx   # Summary card + download buttons
│   ├── package.json
│   └── vite.config.js
├── docker-compose.yml
└── README.md
```

2.3 Technology Stack

Layer	Technology	Version	Purpose
Frontend	React	18.x	Component-based UI framework
Frontend	Vite	5.x	Build tool and dev server
Frontend	Tailwind CSS	3.x	Utility-first styling
Frontend	Axios	1.x	HTTP client for backend calls
Backend	Python	3.11+	Runtime language
Backend	FastAPI	0.110.x	Async REST API framework
Backend	httpx	0.27.x	Async HTTP client for external API calls
Backend	Pydantic	2.x	Data validation and serialisation
Reporting	ReportLab	4.x	PDF generation
Reporting	python-docx	1.x	DOCX generation
DevOps	Docker Compose	2.x	Single-command local deployment

3. Backend — Python FastAPI

3.1 Data Models (analyzer/models.py)

All data in the backend is typed through Pydantic models. This ensures that every finding produced by a check conforms to a known schema before it reaches the API response or report generator. The two core models are Severity and Finding.

```
from pydantic import BaseModel
from enum import Enum
from datetime import datetime
from typing import List, Dict, Any

class Severity(str, Enum):
    CRITICAL = "CRITICAL"
    HIGH     = "HIGH"
    MEDIUM   = "MEDIUM"
    LOW      = "LOW"

class Finding(BaseModel):
    id:              str            # 'F-01' through 'F-06'
    title:           str
    severity:        Severity
    owasp_ref:       str            # e.g. 'OWASP API1:2023'
    endpoint:        str            # affected URL path
    description:     str
    evidence:        Dict[str, Any] # actual request/response proof
    business_impact: str
    remediation:     List[str]      # ordered fix steps

class AnalysisReport(BaseModel):
    target_api:      str
    base_url:        str
    scan_timestamp:  datetime
    analyst:         str
    total_findings:  int
    severity_counts: Dict[str, int] # {'CRITICAL':1, 'HIGH':2, ...}
    overall_risk:    str            # 'HIGH RISK', 'MODERATE RISK', etc.
    findings:        List[Finding]
```

3.2 Security Checks (analyzer/checks/)

Each check is an independent async function that accepts an httpx.AsyncClient and the target base URL, then returns either a populated Finding object or None. Returning None means the check passed — no vulnerability was detected. This design keeps each check completely self-contained and easy to extend.

F-01 — auth_check.py: Unauthenticated Access

- Sends GET /users with no Authorization header.
- Returns a CRITICAL finding if the response is 200 OK and the body contains an array of objects with email or name fields.
- OWASP reference: API2:2023 Broken Authentication.

```
async def check_unauthenticated_access(client, base_url) -> Finding | None:
    resp = await client.get(f'{base_url}/users')
    if resp.status_code == 200:
        data = resp.json()
        if isinstance(data, list) and len(data) > 0:
            if 'email' in data[0] or 'name' in data[0]:
                return Finding(
                    id='F-01', severity=Severity.CRITICAL,
                    owasp_ref="OWASP API2:2023",
                    evidence={'status': 200, 'sample': data[0], 'auth_header':
None},
                    ...)
    return None
```

F-02 — bola_check.py: Broken Object Level Authorization

- Sends GET /users/1, /users/2, and /users/3 sequentially.
- Returns a HIGH finding if all three return 200 OK with different user objects — confirming that any ID can be queried without authorization.
- OWASP reference: API1:2023 Broken Object Level Authorization.

F-03 — data_exposure.py: Excessive Data Exposure

- Sends GET /users/1 and inspects every field in the response JSON.
- Flags fields that have no business use case for typical API consumers: address.geo, company.bs, company.catchPhrase, website.
- Severity is HIGH if 3+ over-exposed fields are found, MEDIUM for 1–2 fields.
- OWASP reference: API3:2023 Broken Object Property Level Authorization.

F-04 — rate_limit_check.py: Missing Rate Limiting

- Sends exactly 5 GET /users requests with a 200 ms pause between each (asyncio.sleep(0.2)).
- Checks every response for rate-limit headers: X-RateLimit-Limit, X-RateLimit-Remaining, Retry-After.
- Returns a MEDIUM finding if none of the 5 responses contain any rate-limit header.
- OWASP reference: API4:2023 Unrestricted Resource Consumption.

F-05 — header_check.py: Missing Security Response Headers

- Sends a single GET /users and inspects the response headers.

- Checks for the presence of: X-Content-Type-Options, X-Frame-Options, Strict-Transport-Security, Content-Security-Policy, Cache-Control.
- Returns a MEDIUM finding listing every absent header.
- OWASP reference: API8:2023 Security Misconfiguration.

F-06 — token_check.py: Auth Token Not Enforced (ReqRes only)

- Performs a POST /api/login to obtain a valid token.
- Then performs GET /api/users without including that token.
- Returns a LOW finding if the second request returns 200 OK with user data.
- This check is skipped automatically when the target is JSONPlaceholder.

3.3 Analysis Engine (analyzer/engine.py)

The engine is the orchestrator. It receives a base URL and analyst name, creates a single shared httpx.AsyncClient, runs all applicable checks sequentially, collects findings, and builds the final AnalysisReport. Sequential execution (rather than concurrent) is intentional — it keeps the request rate low and ensures ethical compliance.

```
async def run_analysis(base_url: str, api_name: str, analyst: str) ->
AnalysisReport:
    findings = []
    async with httpx.AsyncClient(timeout=10.0) as client:
        for check_fn in get_applicable_checks(base_url):
            result = await check_fn(client, base_url)
            if result:
                findings.append(result)

    counts = count_by_severity(findings)
    return AnalysisReport(
        target_api=api_name,
        base_url=base_url,
        scan_timestamp=datetime.utcnow(),
        analyst=analyst,
        total_findings=len(findings),
        severity_counts=counts,
        overall_risk=calculate_risk(counts),
        findings=findings
    )

def calculate_risk(counts: dict) -> str:
    if counts.get("CRITICAL", 0) > 0: return "HIGH RISK"
    if counts.get("HIGH", 0) > 0:     return "ELEVATED RISK"
    if counts.get("MEDIUM", 0) > 0:   return "MODERATE RISK"
    return "LOW RISK"
```

3.4 API Endpoints (main.py)

Method	Path	Description	Response
GET	/api/health	Health check — confirms backend is running	200 {status: ok}
POST	/api/analyze	Runs the full analysis against the specified target API	AnalysisReport JSON
GET	/api/report/pdf	Generates and returns the PDF report for download	application/pdf
GET	/api/report/docx	Generates and returns the DOCX report for download	application/vnd.openxmlformats...

```
# POST /api/analyze — request body schema
{
  "api_name": "JSONPlaceholder",
  "base_url": "https://jsonplaceholder.typicode.com",
  "analyst":  "CrazyCoder"
}
```

> **NOTE**
> CORS is enabled for http://localhost:3000. In production, replace the allow_origins list with the actual deployed frontend domain.

4. Frontend — React + Vite

4.1 Component Architecture

The frontend is a single-page application with five components arranged in a two-panel layout. The Dashboard component owns all shared state and passes data down to children via props. No external state management library is used — React's built-in useState is sufficient for the application's data flow.

Component	File	Responsibility
Dashboard	Dashboard.jsx	Root layout, scan state, coordinates all children
ApiSelector	ApiSelector.jsx	Renders target API cards, captures analyst name input
ProgressTracker	ProgressTracker.jsx	Animates the six check steps during an active scan
FindingsTable	FindingsTable.jsx	Tabular findings with severity filter and expandable detail rows
SeverityBadge	SeverityBadge.jsx	Color-coded pill label (CRITICAL / HIGH / MEDIUM / LOW)
ReportDownload	ReportDownload.jsx	Summary card with severity counts and PDF/DOCX download buttons

4.2 State Management

All application state lives in Dashboard.jsx and flows downward. The five key state variables are:

```
const [selectedApi,  setSelectedApi]  = useState(null);     // {name, baseUrl}
const [analystName,  setAnalystName]  = useState('');       // free-text input
const [isScanning,   setIsScanning]   = useState(false);    // controls spinner
const [report,       setReport]       = useState(null);     // AnalysisReport |
null
const [error,        setError]        = useState(null);     // error message |
null
```

4.3 API Client (api.js)

All calls to the FastAPI backend are centralised in a single Axios instance. This keeps base URL configuration in one place and makes it easy to add interceptors for error handling or authentication in future.

```
import axios from 'axios';

const apiClient = axios.create({
```

```
    baseURL: 'http://localhost:8000',
    timeout: 30000,
    headers: { 'Content-Type': 'application/json' }
  });

  export const runAnalysis = (payload) =>
    apiClient.post('/api/analyze', payload).then(r => r.data);

  export const downloadReport = (format, apiName) =>
    apiClient.get(`/api/report/${format}`, {
      params: { api_name: apiName },
      responseType: 'blob'
    });
```

4.4 Routing & Layout

The application has a single route. The two-panel layout uses Tailwind's flex utilities: the left panel (30% width) contains the ApiSelector and scan controls; the right panel (70%) conditionally renders either the ProgressTracker during a scan or the FindingsTable and ReportDownload after a scan completes.

5. Security Checks Reference

The table below is the canonical reference for all six checks implemented in version 1.0.0. Each row maps the internal finding ID to its OWASP category, affected endpoint, severity, and whether it applies to both target APIs or only one.

ID	Title	Severity	OWASP Ref	Target	Endpoint(s)
F-01	Unauthenticated Access	CRITICAL	API2:2023	Both	GET /users
F-02	Broken Object Level Auth (BOLA)	HIGH	API1:2023	Both	GET /users/{1,2,3}
F-03	Excessive Data Exposure	HIGH	API3:2023	Both	GET /users/1
F-04	Missing Rate Limiting	MEDIUM	API4:2023	Both	GET /users (×5)
F-05	Missing Security Headers	MEDIUM	API8:2023	Both	GET /users
F-06	Token Not Enforced	LOW	API2:2023	ReqRes	POST /api/login + GET /api/users

6. Installation and Setup

6.1 Prerequisites

Requirement	Minimum Version	Check Command
Python	3.11	python --version
Node.js	18.x	node --version
npm	9.x	npm --version
Docker	24.x	docker --version (optional)
Postman	10.x	Desktop app — postman.com

6.2 Backend Setup

1. Clone or download the project repository.
2. Navigate to the backend directory: cd api-security-analyzer/backend
3. Create a virtual environment: python -m venv venv
4. Activate the environment: source venv/bin/activate (Linux/Mac) or venv\Scripts\activate (Windows)
5. Install dependencies: pip install -r requirements.txt
6. Start the development server: uvicorn main:app --reload --port 8000
7. Verify at http://localhost:8000/api/health — should return {"status": "ok"}.

```
# requirements.txt
fastapi==0.110.0
uvicorn==0.29.0
httpx==0.27.0
pydantic==2.6.0
reportlab==4.1.0
python-docx==1.1.0
jinja2==3.1.3
python-multipart==0.0.9
```

6.3 Frontend Setup

8. In a new terminal, navigate to the frontend directory: cd api-security-analyzer/frontend
9. Install dependencies: npm install
10. Start the development server: npm run dev
11. Open http://localhost:3000 in your browser.

TIP

> The frontend proxies API requests to localhost:8000 via Vite's proxy config. Both servers must be running simultaneously for the application to function.

6.4 Docker Compose (Single-Command)

For a single-command startup, use Docker Compose from the project root:

```
# Start both services
docker-compose up --build

# Stop all services
docker-compose down

# Services exposed:
#   Frontend  → http://localhost:3000
#   Backend   → http://localhost:8000
#   API Docs  → http://localhost:8000/docs
```

7. Usage Guide

7.1 Running a Security Analysis

12. Open the application at http://localhost:3000.
13. In the left panel, click one of the two API cards to select a target: JSONPlaceholder or ReqRes.
14. Enter your name in the Analyst Name field.
15. Click Run Security Analysis. The ProgressTracker panel will appear on the right, showing each of the six checks transitioning from Pending to In Progress to Complete.
16. When the scan completes (typically 10–20 seconds), the FindingsTable will render with all identified vulnerabilities.
17. Click any row to expand the full finding details, including the evidence block and remediation steps.
18. Use the severity filter buttons above the table to narrow the view to CRITICAL, HIGH, MEDIUM, or LOW findings.

7.2 Downloading the Report

After a scan completes, the ReportDownload section appears above the findings table. It displays a summary card showing the overall risk rating, total finding count, and a severity breakdown. Two buttons provide immediate downloads:

- Download PDF Report — triggers GET /api/report/pdf and downloads a multi-page PDF styled for professional client delivery.
- Download Word Report — triggers GET /api/report/docx and downloads an equivalent Word document that can be edited or incorporated into a larger deliverable.

7.3 Testing with Postman Directly

The backend can also be tested independently of the frontend using Postman. Import the collection or build requests manually:

```
# Health check
GET http://localhost:8000/api/health

# Run a full analysis
POST http://localhost:8000/api/analyze
Content-Type: application/json
{
  "api_name": "JSONPlaceholder",
  "base_url": "https://jsonplaceholder.typicode.com",
  "analyst":  "CrazyCoder"
}

# Download PDF (after analysis populates the report cache)
```

```
GET http://localhost:8000/api/report/pdf?api_name=JSONPlaceholder
```

8. Generated Report Format

Both the PDF and DOCX outputs follow an identical eight-section structure modelled on professional AppSec agency deliverables. The backend generates both from the same AnalysisReport object using separate renderer functions.

Section	Title	Primary Audience	Contents
1	Cover Page	All	Project name, target API, analyst, date, overall risk rating
2	Executive Summary	Management	Risk narrative, finding count by severity, top-priority action
3	Scope & Methodology	Technical leads	APIs tested, tools used, test types performed, ethical boundaries
4	API Overview	Developers	Endpoint inventory, authentication mechanism, data formats
5	Findings Summary Table	All	One row per finding: ID, title, severity, endpoint, brief description
6	Detailed Findings	Developers	Full DEBA structure per finding: Description, Evidence, Business Impact, Action
7	Remediation Roadmap	Engineering	Prioritised fix list with severity, effort, and recommended action
8	Conclusion	All	Overall security posture assessment and closing commentary

Within Section 6, every finding follows the DEBA structure: Description (what the vulnerability is), Evidence (the actual HTTP request and response proving it exists), Business Impact (risk in plain business language), and Action (specific, implementable remediation steps).

9. Extending the Project

9.1 Adding a New Security Check

New checks slot cleanly into the existing architecture. The steps below add a hypothetical F-07 check for verbose error messages:

19. Create backend/analyzer/checks/verbose_errors.py.
20. Define an async function with the standard signature: async def check_verbose_errors(client, base_url) -> Finding | None.
21. Implement the check logic — send a malformed request and inspect the error response body for stack traces or internal paths.
22. Return a populated Finding or None.
23. Register the check in engine.py: add it to the checks list returned by get_applicable_checks().
24. Add the new finding ID and OWASP mapping to the frontend's ProgressTracker component's steps array.

9.2 Adding a New Target API

To permit a new public demo API as an analysis target:

25. Add the domain to the ALLOWED_DOMAINS list in main.py.
26. Review each existing check to confirm the endpoint paths are compatible, or implement conditional logic using the base_url parameter.
27. Add a new card object to the APIs array in ApiSelector.jsx with the name, URL, description, and applicable check count.

9.3 Adding a New Report Format

The report generator in report/generator.py is designed to accept the same AnalysisReport object and produce different output formats. Adding HTML export, for example, involves writing a new generate_html(report) function using Jinja2 templating and registering a new GET /api/report/html endpoint in main.py.

10. Ethical Compliance

Ethical compliance is enforced at multiple layers of the application — it is not merely a policy statement. The mechanisms below are present in the codebase and cannot be bypassed through normal use of the application.

10.1 Allow-List Enforcement

The backend maintains a hard-coded ALLOWED_DOMAINS set. Any POST /api/analyze request whose base_url resolves to a domain outside this set receives an immediate HTTP 403 Forbidden response. The analysis engine is never invoked.

```
ALLOWED_DOMAINS = {
    "jsonplaceholder.typicode.com",
    "reqres.in"
}

def validate_target(base_url: str):
    from urllib.parse import urlparse
    domain = urlparse(base_url).netloc
    if domain not in ALLOWED_DOMAINS:
        raise HTTPException(
            status_code=403,
            detail='Target not in approved scope. Educational use only.'
        )
```

10.2 Rate-Limit Check Constraints

The F-04 rate-limit check sends a maximum of 5 requests with a mandatory 200 ms pause between each. This is enforced in code via asyncio.sleep(0.2) and a loop that terminates at iteration 5. There is no mechanism to increase this limit through normal use.

10.3 Read-Only Methods

The httpx client in the analysis engine is configured to use only GET and safe POST requests (the login endpoint for F-06). No PUT, PATCH, or DELETE methods are called at any point in the codebase. The frontend UI disclaimer banner is always visible and cannot be dismissed.

10.4 Summary of Constraints

Constraint	Enforcement Location	What It Prevents
Allow-list domain check	main.py — before engine invocation	Testing any non-approved API

Max 5 requests in F-04	rate_limit_check.py — loop limit	Accidental DoS or flooding
200ms delay between requests	rate_limit_check.py — asyncio.sleep	High-rate request bursts
10-second HTTP timeout	httpx.AsyncClient constructor	Hanging requests to slow targets
GET/POST only	All check files — no other methods	Modifying or deleting data
UI disclaimer banner	Dashboard.jsx — always rendered	Misrepresentation of tool purpose

11. Known Limitations

The following limitations are acknowledged in version 1.0.0. They represent deliberate scoping decisions for the internship context rather than defects.

- Report cache: The generated report is stored in memory for the duration of the server session. Restarting the backend clears all cached reports. A production implementation would persist reports to a database or object store.
- Single concurrent scan: The engine does not support parallel analysis of multiple APIs simultaneously. Only one scan can be active per server process.
- No authentication on the tool itself: The FastAPI application has no user authentication. Anyone with network access to port 8000 can trigger an analysis. This is acceptable in a local development context but would require an auth layer before any internet-facing deployment.
- Demo API instability: JSONPlaceholder and ReqRes are third-party public services. Temporary unavailability of these services will cause checks to time out and return null findings rather than flagging an error to the user.
- F-04 observational only: The rate-limiting check can only confirm the absence of rate-limit headers. It cannot definitively prove that rate limiting is entirely absent — the server may enforce limits at a lower layer without advertising them in headers.

12. OWASP API Security Top 10 Reference

The table below maps each OWASP API Security Top 10 (2023) category to its coverage status in this project version.

ID	Vulnerability	Covered?	Finding ID
API1:2023	Broken Object Level Authorization	Yes	F-02
API2:2023	Broken Authentication	Yes	F-01, F-06
API3:2023	Broken Object Property Level Auth.	Yes	F-03
API4:2023	Unrestricted Resource Consumption	Yes	F-04
API5:2023	Broken Function Level Authorization	Partial	Scope review only
API6:2023	Unrestricted Access to Sensitive Flows	No	Out of scope v1.0
API7:2023	Server Side Request Forgery (SSRF)	No	Out of scope v1.0
API8:2023	Security Misconfiguration	Yes	F-05
API9:2023	Improper Inventory Management	Partial	Documentation review
API10:2023	Unsafe Consumption of APIs	No	Out of scope v1.0

13. Changelog

v1.0.0 — Initial Release

- Full-stack implementation: FastAPI backend + React + Vite frontend.
- Six security checks: F-01 through F-06, covering OWASP API1–API4 and API8.
- Two target APIs: JSONPlaceholder and ReqRes.
- PDF report generation via ReportLab.
- DOCX report generation via python-docx.
- Docker Compose configuration for single-command local deployment.
- Allow-list domain enforcement and read-only constraint in all checks.

Planned for v1.1.0

- F-07: Verbose error message detection (stack traces in 500 responses).
- F-08: JWT algorithm confusion check on ReqRes tokens.
- Persistent report storage using SQLite.
- Report export to HTML.
- Support for user-supplied custom API targets (with manual approval gate).

Appendix A: Environment Variables

The following environment variables can be set to override defaults. All have sensible defaults for local development and do not need to be set unless deploying to a non-local environment.

Variable	Default	Description
BACKEND_HOST	0.0.0.0	Host the FastAPI server binds to
BACKEND_PORT	8000	Port the FastAPI server listens on
FRONTEND_PORT	3000	Port the Vite dev server listens on
CORS_ORIGINS	http://localhost:3000	Comma-separated list of allowed CORS origins
REQUEST_TIMEOUT	10	Seconds before an outbound HTTP request times out
MAX_RATE_REQUESTS	5	Maximum requests sent during the F-04 rate-limit check

Appendix B: Glossary

Term	Definition
API	Application Programming Interface — a defined contract for communication between two software systems.
AppSec	Application Security — the discipline of building and auditing applications to resist attack.
BOLA	Broken Object Level Authorization — a vulnerability allowing access to other users' resources by changing a resource identifier.
CORS	Cross-Origin Resource Sharing — a browser mechanism controlling which domains can call a given API.
DEBA	Description / Evidence / Business Impact / Action — the standard structure for a security finding write-up.
FastAPI	A modern Python web framework with native async support and automatic OpenAPI documentation generation.
httpx	An async Python HTTP client library used to make outbound requests within the analysis checks.
JWT	JSON Web Token — a compact, self-contained token format used for authentication, verified without a database lookup.
OWASP	Open Web Application Security Project — the non-profit that publishes the globally recognised API Security Top 10.
PII	Personally Identifiable Information — any data that can directly identify an individual.
Pydantic	A Python data validation library that enforces type constraints on data structures at runtime.
Rate Limiting	A server-side control that caps the number of requests a client may make within a defined time window.
REST	Representational State Transfer — an architectural style for APIs using HTTP methods and URL-identified resources.
SSRF	Server-Side Request Forgery — a vulnerability allowing an attacker to induce the server to make HTTP requests to arbitrary destinations.
Vite	A fast JavaScript build tool and development server used to bundle the React frontend.

— End of Documentation —

API Security Risk Analyzer · v1.0.0 · Cybersecurity Internship Task 3